

Course Project Proposal

Dockerized Domain-Specific AI Agent

Dr. Mohamad Aoude
Faculty of Engineering

1 Project Overview

Students will work in teams of three to build a working, domain-specific AI agent. Each team will select an application area, define a realistic user problem, and implement an agent that can answer questions, analyze structured inputs, use tools, maintain state, and complete a useful workflow.

The complete application must run in Docker so that it can be started and evaluated consistently on another computer.

The system must:

- answer grounded questions using an approved domain knowledge source;
- analyze user input and produce an appropriate recommendation or action;
- invoke at least four domain-relevant tools;
- maintain conversation context and task state;
- handle ambiguous or unsupported requests safely; and
- provide a usable web interface running inside Docker.

2 Possible Application Areas

Teams may propose one of the following applications or submit another idea for instructor approval. Application areas are grouped into three categories to help teams locate a domain that matches their background.

Industrial and Engineering Applications

1. **Industrial Equipment Maintenance Assistant:** Analyzes alarms, operating conditions, and maintenance history to suggest inspections, classify fault severity, and generate a structured work-order report.
2. **Digital Twin Monitoring Agent:** Connects to the structured state data of a digital twin (provided as JSON or CSV snapshots), detects anomalies against expected operating envelopes, explains deviations, and recommends corrective or preventive actions.
3. **Smart Manufacturing Quality Agent:** Receives production-line sensor readings and inspection results, classifies defect types, identifies likely root causes, and generates a structured quality report for process engineers.
4. **Process Optimization Advisor:** Analyzes operating parameters from a chemical, food, or utility process (provided as structured records), detects off-spec conditions, and recommends parameter adjustments within predefined safe bounds.

5. **Building and Infrastructure Management Agent:** Monitors structured sensor data from a smart building or civil infrastructure asset (HVAC, power, occupancy), detects faults, prioritizes maintenance tasks, and generates a facility report.
6. **Vehicle Diagnostic Assistant:** Collects symptoms, vehicle information, and diagnostic codes; proposes likely causes; recommends safe checks; and generates a mechanic-ready report.
7. **Energy-Consumption Advisor:** Analyzes usage records, detects unusual patterns, identifies likely causes, and recommends efficiency improvements with estimated impact.

Business, Operations, and IT

8. **Project Management Assistant:** Creates tasks, assigns priorities, identifies risks and dependencies, tracks status, and produces progress summaries.
9. **IT Help-Desk Diagnostic Agent:** Troubleshoots network, operating-system, account, or application problems through a structured diagnostic workflow and generates a resolution report.
10. **Logistics and Delivery Coordinator:** Tracks orders, identifies delays, prioritizes deliveries, proposes re-routing alternatives, and generates status reports.
11. **Cybersecurity Triage Assistant:** Classifies reported security events, recommends containment steps, and creates structured incident reports for the operations team.
12. **Supply Chain Risk Agent:** Monitors structured supplier and inventory records, identifies disruption risks (lead-time variance, stock shortfall, single-source exposure), and recommends mitigation actions.
13. **Customer Support Agent:** Handles grounded questions about a simulated business (restaurant, clinic, hotel, or online shop), checks availability, creates and cancels bookings, and maintains conversation state across turns.

Health, Environment, and Education

14. **Health Information and Triage Assistant:** Collects symptoms, provides general educational guidance, identifies warning signs, and recommends an appropriate level of professional care.
15. **Agricultural Diagnostic Assistant:** Uses reported crop symptoms and environmental conditions to suggest possible causes, safe observations, and escalation to an agricultural specialist.
16. **Environmental Monitoring Agent:** Ingests structured sensor readings (air quality, water quality, or weather stations), detects threshold violations, classifies event severity, and generates a structured alert or regulatory report.
17. **Educational Support Agent:** Diagnoses learning gaps from student answers, recommends targeted practice, tracks progress across sessions, and generates a personalized study plan.

Restriction for Safety-Critical Domains

Projects involving health, vehicles, industrial equipment, digital twins, cybersecurity, or other safety-critical areas must be presented as **decision-support or triage systems**, not authoritative diagnostic systems. They must communicate uncertainty, avoid unsupported claims, provide clear limitations, state the source of every recommendation, and escalate high-risk cases to a qualified professional.

Note on Digital Twin Projects

Teams choosing a digital twin topic do not need to build or simulate the physical asset. They must define a realistic structured data source (JSON or CSV snapshots representing asset state), implement the four required tools against that source, and explain in their report which real-world integration would replace the static files in a production deployment.

3 Learning Objectives

The project applies the principal concepts covered throughout the course:

- system layers and the limitations of standalone LLMs;
- grounding and retrieval-evaluation concepts;
- agent loops, tool use, and task completion;
- workflow selection, controls, and observability;
- short-term, working, and optional long-term memory;
- typed information and action tools;
- framework, model, and infrastructure selection; and
- evaluation, safety, and production-oriented architecture.

Project Scope

The project does not require a retrieval-augmented generation pipeline, vector database, or document embeddings. Domain information will be small and structured, so it must be stored in JSON, YAML, CSV, or SQLite and accessed through deterministic tools.

Teams must explain why this design is appropriate and identify conditions under which RAG would become useful.

4 System Architecture

The proposed system contains the following layers:

1. **Chat interface:** A Gradio, Streamlit, or equivalent interface.
2. **Orchestration layer:** The agent workflow, routing logic, stopping conditions, and state transitions.
3. **LLM reasoning core:** Interprets requests and selects the next permitted action.
4. **Tool layer:** Typed functions for domain lookup, analysis, actions, and reporting.
5. **Memory layer:** Conversation context and explicit task state.
6. **Data layer:** Structured domain data and a SQLite database when persistent records are required.
7. **Container layer:** A Docker image containing the application, runtime, dependencies, and startup configuration.

5 Required Tools

Each tool must have a documented purpose, input schema, output schema, and error behavior. Every project must implement at least four tools covering the following categories:

1. **Information Tool:** Retrieves grounded facts, policies, procedures, specifications, or domain rules.
2. **Analysis Tool:** Validates inputs, calculates a score, classifies a case, checks constraints, or applies deterministic diagnostic rules.
3. **Action Tool:** Creates or updates an authorized record, task, incident, maintenance request, or other domain object.
4. **Reporting Tool:** Produces a structured summary, recommendation, checklist, handoff report, or exportable result.

Additional tools are allowed when they add clear value. Sensitive or state-changing actions must require explicit user confirmation.

6 Memory Requirements

6.1 Short-Term Memory

Short-term memory is mandatory. It must preserve the current conversation, including the user's name and recent context, within the active session.

6.2 Working Memory

Working memory is mandatory. It must explicitly represent:

- the current intent;
- information already collected from the user;
- missing required fields;
- pending confirmation;
- the latest tool result; and
- the current workflow state.

6.3 Long-Term Memory

Long-term memory is optional and eligible for bonus credit. It may store returning-user preferences across separate sessions in a file or database.

7 Workflow and Control Requirements

Teams must select and justify a workflow that matches the task. A recommended baseline is a router that classifies the request by intent, followed by a bounded sequence of information gathering, analysis, tool execution, safety checking, and response generation.

The implementation must:

- define explicit workflow states and transitions;
- set a maximum number of model and tool iterations;
- validate all tool arguments before execution;
- require confirmation before sensitive or state-changing actions;
- log intents, tool calls, tool results, errors, and fallback events; and

- use a deterministic fallback when routing confidence is insufficient.

For unsupported requests, the agent must state its limitation, avoid inventing an answer, and offer a simulated human-handoff path.

8 Team Roles

Student	Role	Responsibilities
Student 1	Tools Engineer	Typed tools, validation, domain data, and database schema.
Student 2	Agent Engineer	Workflow, routing, prompts, stopping rules, and fallback logic.
Student 3	Platform and Interface	Memory/state, web interface, Docker packaging, trace logging, end-to-end tests, and demo preparation.

All students must understand the complete system and may be questioned about any component.

9 Three-Week Timeline

Week	Milestone
Week 1	Application proposal and architecture approved. Domain data, database, and four tools implemented and tested independently.
Week 2	Workflow integrated. Memory, confirmation gates, stopping rules, and fallback behavior operational. Initial Docker image builds successfully.
Week 3	Dockerized interface completed. Evaluation suite executed, traces analyzed, report written, and live demonstration rehearsed.

10 Deliverables

1. A working demonstration launched through Docker.
2. Source code, dependency file, setup instructions, and an example environment file without secrets.
3. A `Dockerfile` and, when multiple services are used, a `compose.yaml` file.
4. Structured domain data and a documented database schema, when used.
5. Four implemented and tested tools.
6. Conversation memory and explicit workflow state.
7. A technical report describing the architecture, workflow choice, controls, framework/model choices, limitations, and team contributions.
8. An evaluation package containing test cases, expected and actual results, summary metrics, and selected execution traces.

11 Minimum Evaluation Suite

Each team must run at least 20 documented test conversations covering:

- grounded domain-information questions;
- valid and invalid analysis inputs;
- successful tool actions and rejected unsafe actions;
- memory across multiple conversation turns;
- missing or ambiguous information;
- unsupported requests;
- duplicate or conflicting actions; and
- attempted prompt or tool misuse.

The evaluation report must include:

- task-completion rate;
- correct tool-selection rate;
- fallback accuracy; and
- the number of unsafe or invalid actions executed.

12 Grading Criteria

Component	Weight
Workflow and reasoning: routing, state, fallback, and stopping	20%
Tool correctness, validation, and data integrity	20%
Memory and state integration	15%
Evaluation, safety controls, and observability	15%
Docker packaging and reproducible execution	10%
Interface and usability	5%
Technical report	5%
Live demonstration, individual understanding, and Q&A	10%
Total	100%

13 Bonus Points

Teams may earn up to 10 bonus points for meaningful extensions such as:

- long-term memory across separate sessions;
- an additional useful tool, such as rescheduling or calendar export;
- multilingual interaction;
- a secured public deployment with secrets excluded; or
- advanced evaluation through an automated dashboard or adversarial tests.

14 Technical and Academic Rules

- Do not use a RAG pipeline, vector database, or document embeddings.
- Store domain information in structured JSON, YAML, CSV, or SQLite data.

- Use an orchestration framework such as LangGraph, Semantic Kernel, AutoGen, CrewAI, or an instructor-approved equivalent.
- The complete project must build and run with Docker.
- The evaluator must be able to start the project with one documented command, preferably `docker compose up -build`.
- Use a non-root container user where practical, expose only required ports, and persist necessary data through a named volume.
- Add a container health check and document all required environment variables.
- Do not hard-code API keys, passwords, or personal information.
- Validate, confirm, and log every sensitive or state-changing action.
- Submit all code in one GitHub repository or ZIP archive.
- Declare the use of AI development tools in the technical report.
- Every student must be able to explain their contribution and the overall system.

15 Expected Outcome

At the end of the project, each team should have a working and explainable AI application that runs reproducibly in Docker and demonstrates practical experience with workflows, tools, memory, evaluation, and explicit safety controls.